



US 20250274394A1

(19) **United States**

(12) **Patent Application Publication**
Schornig et al.

(10) **Pub. No.: US 2025/0274394 A1**

(43) **Pub. Date: Aug. 28, 2025**

(54) **QOE-DRIVEN PREDICTIVE NETWORKS**

(71) Applicant: **Cisco Technology, Inc.**, San Jose, CA
(US)

(72) Inventors: **Eduard Schornig**, Haarlem (NL);
Jean-Philippe Vasseur, Combloux
(FR); **Grégory Mermoud**, Venthône
(CH); **Pierre-André Savalle**,
Rueil-Malmaison (FR)

(73) Assignee: **Cisco Technology, Inc.**, San Jose, CA
(US)

(21) Appl. No.: **18/585,423**

(22) Filed: **Feb. 23, 2024**

Publication Classification

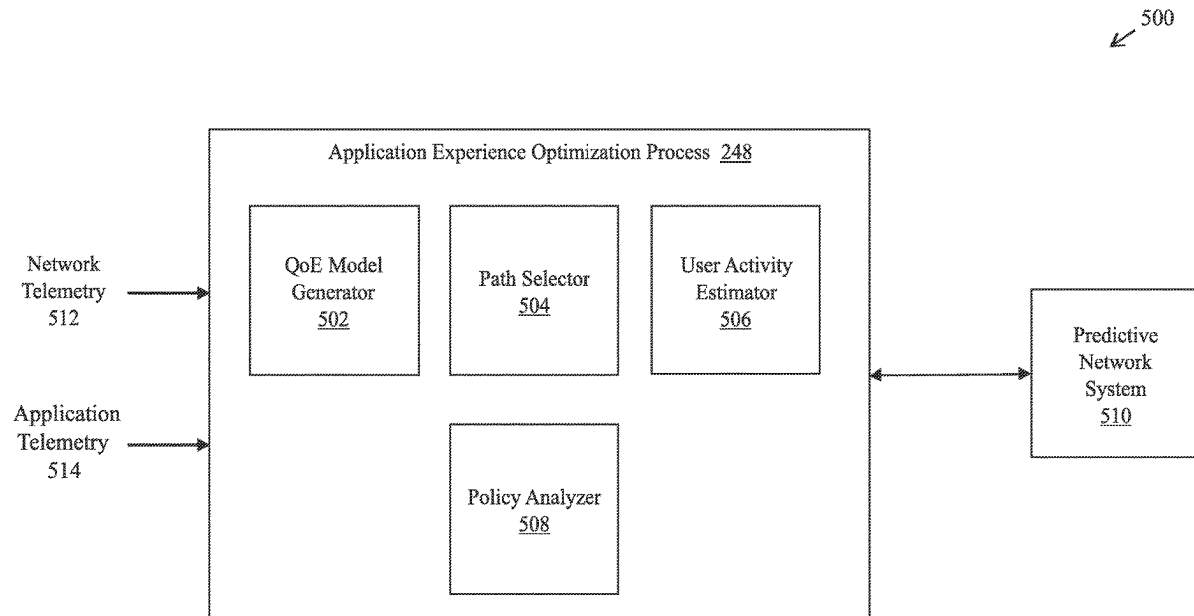
(51) **Int. Cl.**
H04L 47/127 (2022.01)
H04L 45/302 (2022.01)
H04L 47/122 (2022.01)

(52) **U.S. Cl.**

CPC **H04L 47/127** (2013.01); **H04L 45/306**
(2013.01); **H04L 45/308** (2013.01); **H04L**
47/122 (2013.01)

(57) **ABSTRACT**

In one embodiment, a device trains a quality of experience model to predict a quality of experience metric for an online application. The device augments a predictive network system previously deployed to a network to use the quality of experience model instead of a prediction model that predicts performance of a network path. The device obtains policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model. The device ensures, based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application.



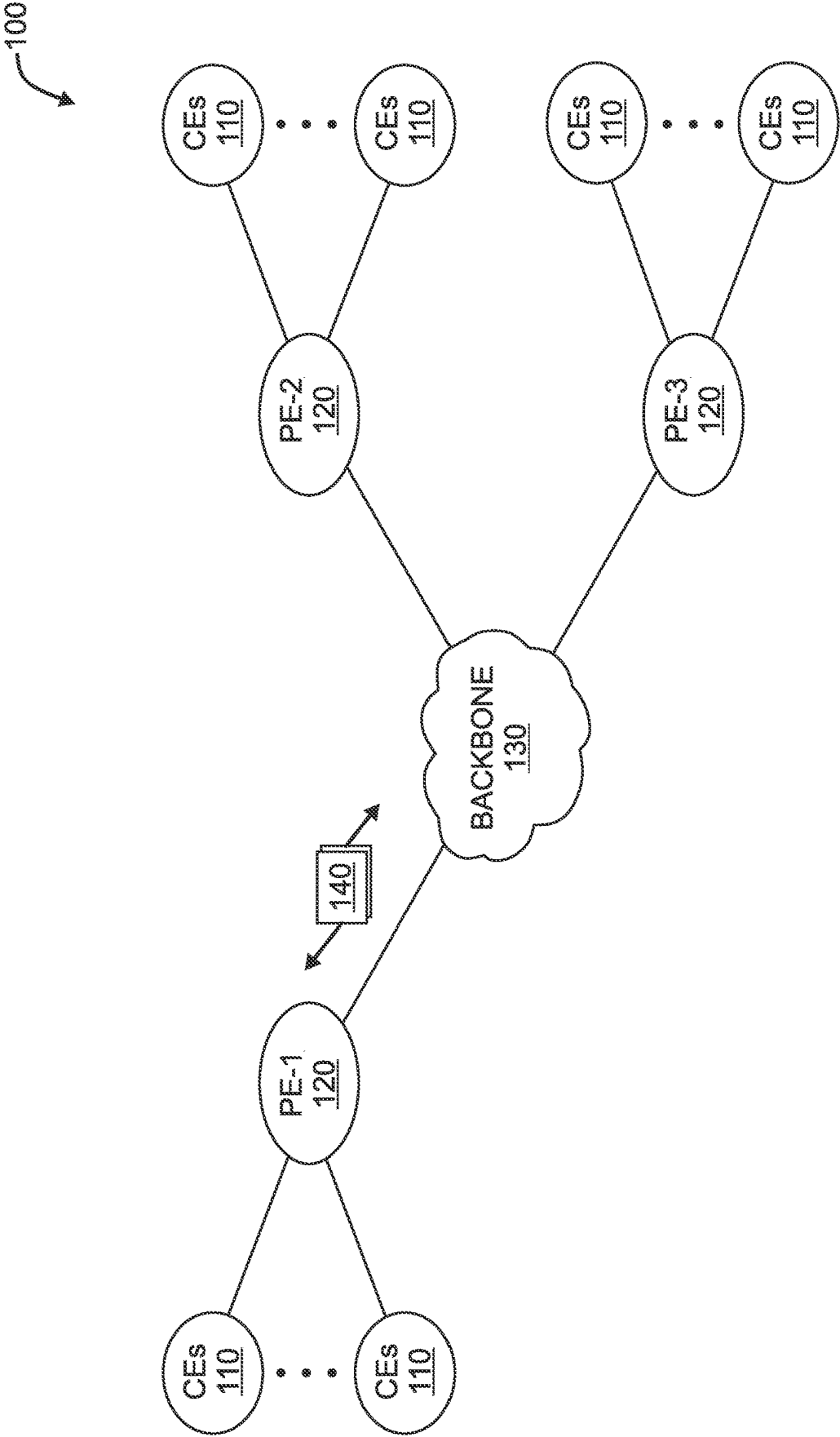


FIG. 1A

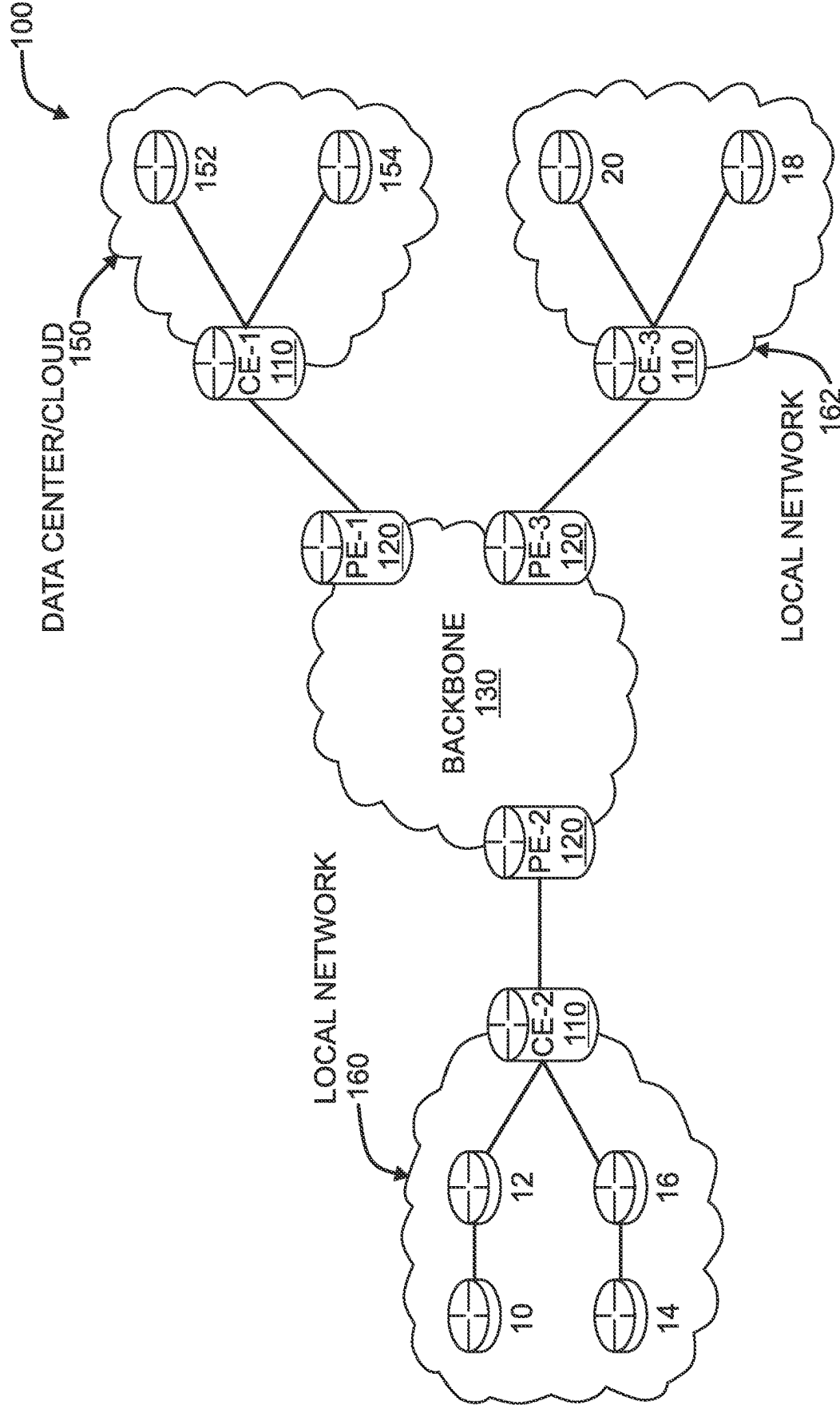


FIG. 1B

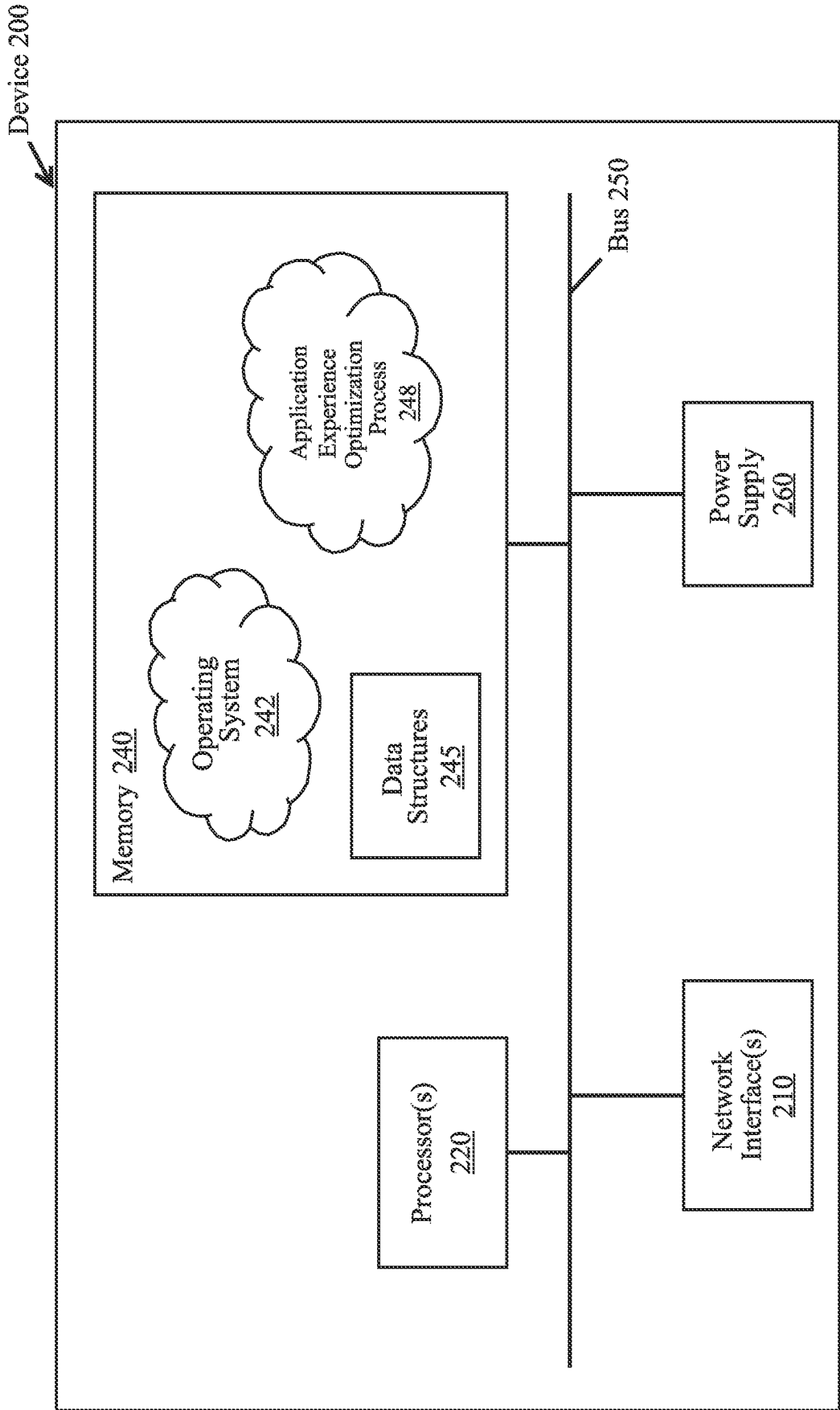


FIG. 2

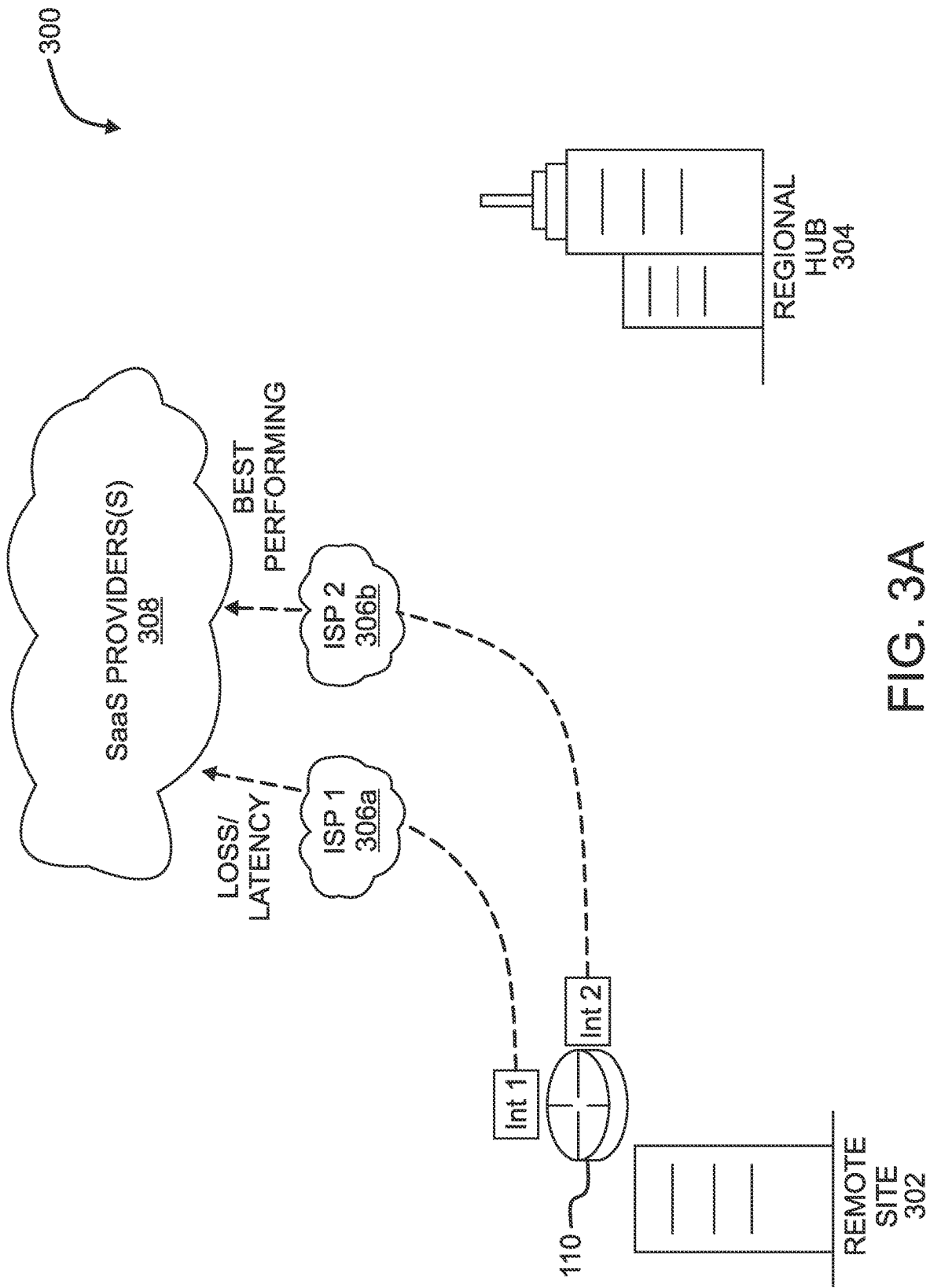


FIG. 3A

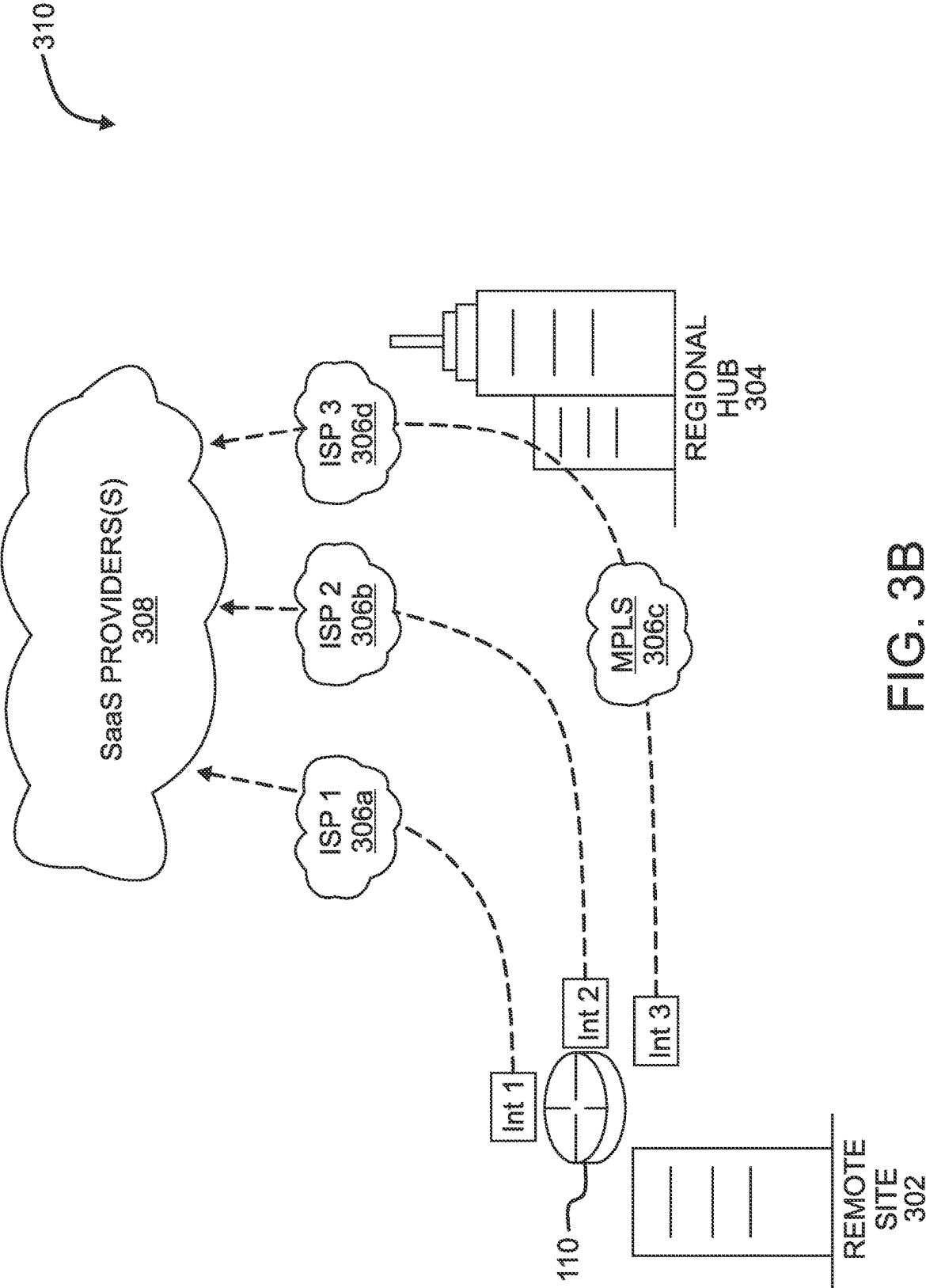


FIG. 3B

400

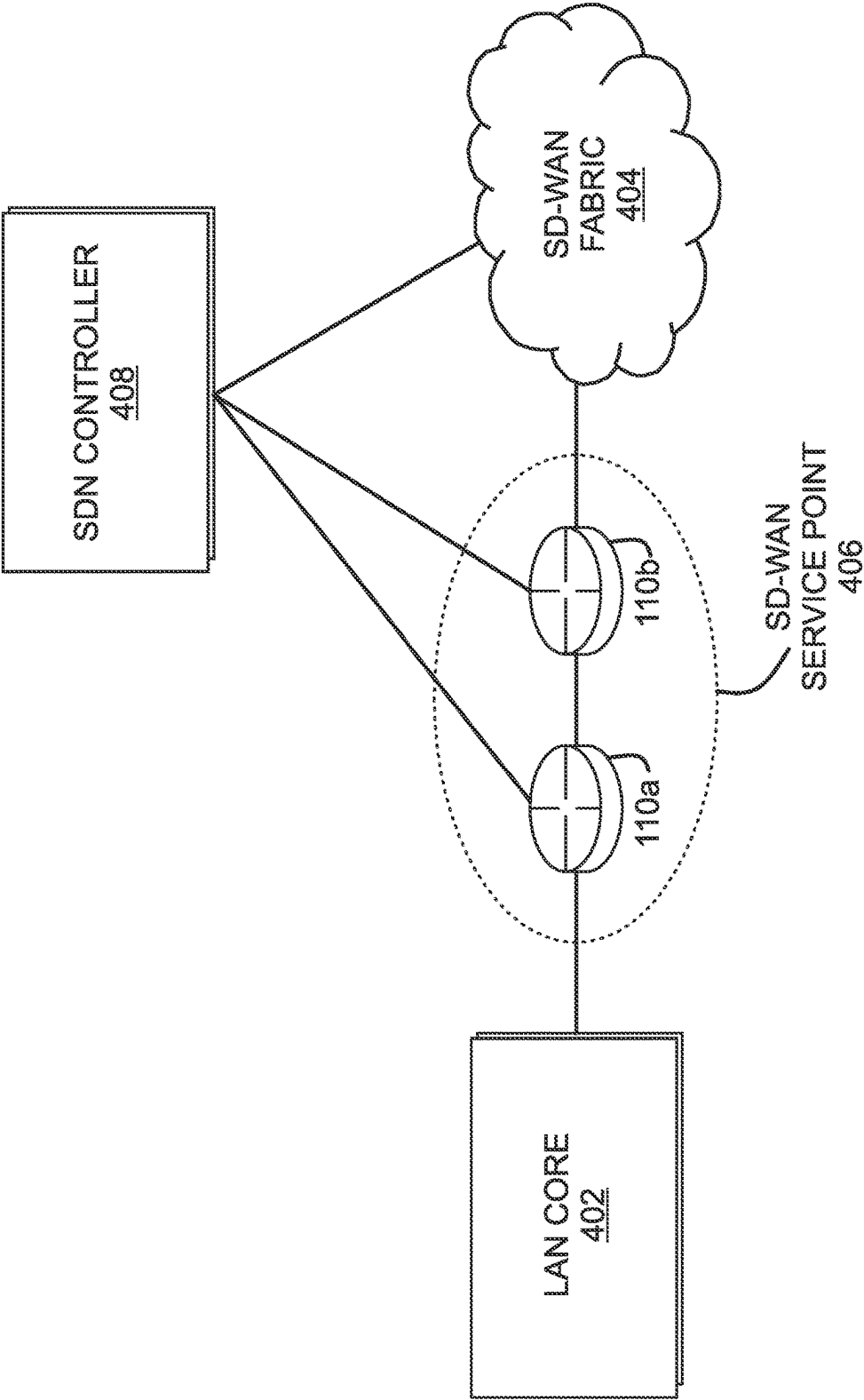


FIG. 4A

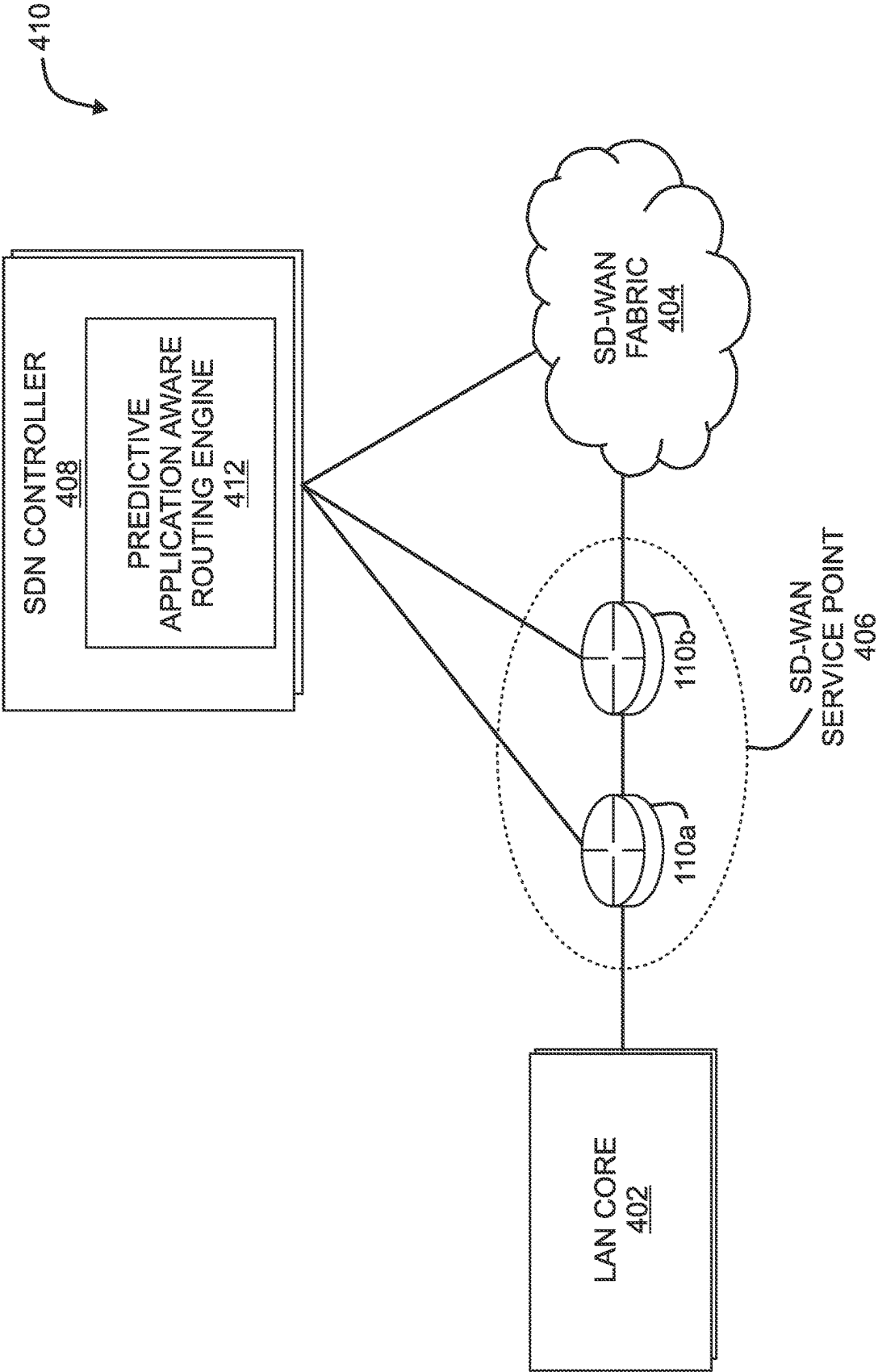


FIG. 4B

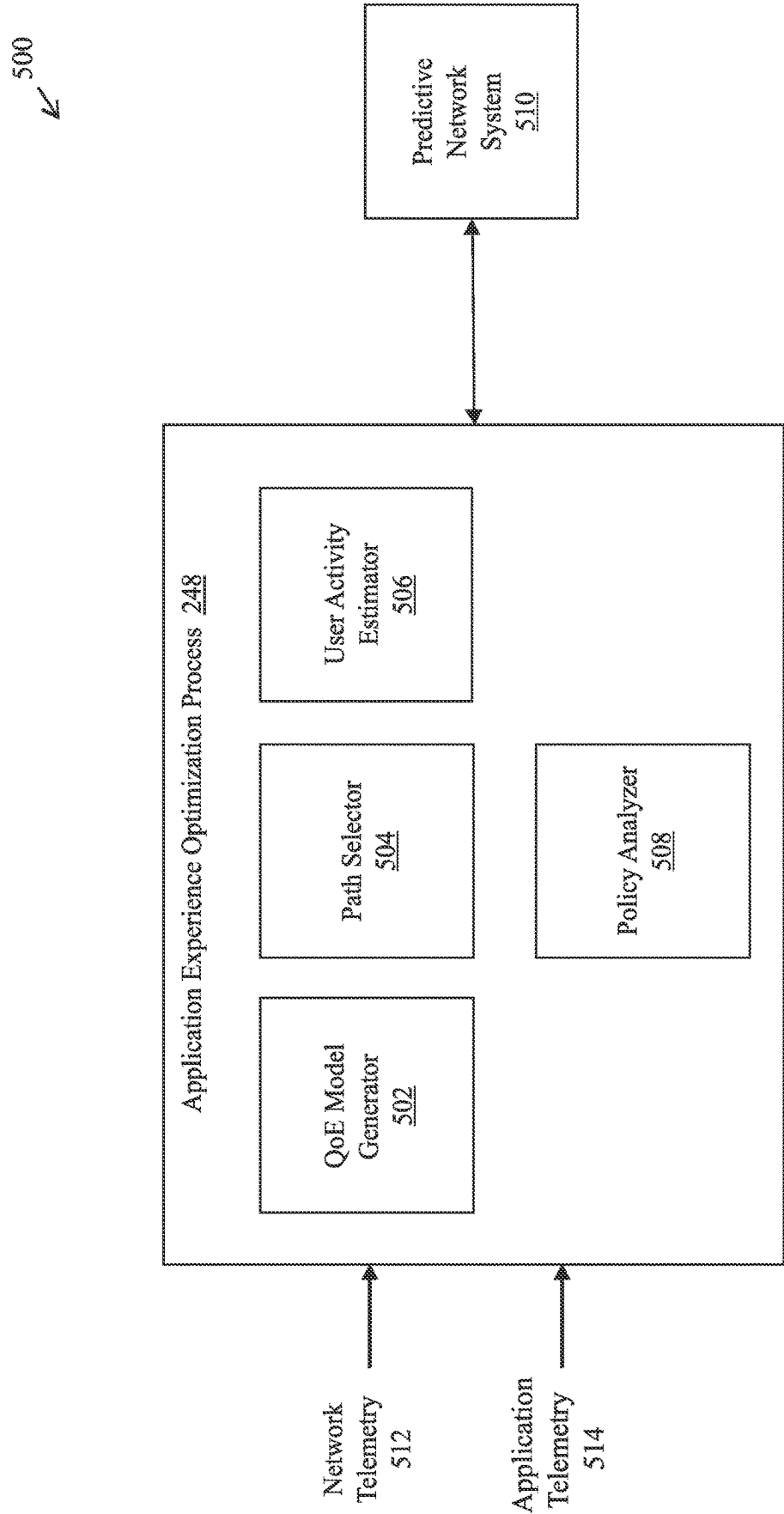


FIG. 5

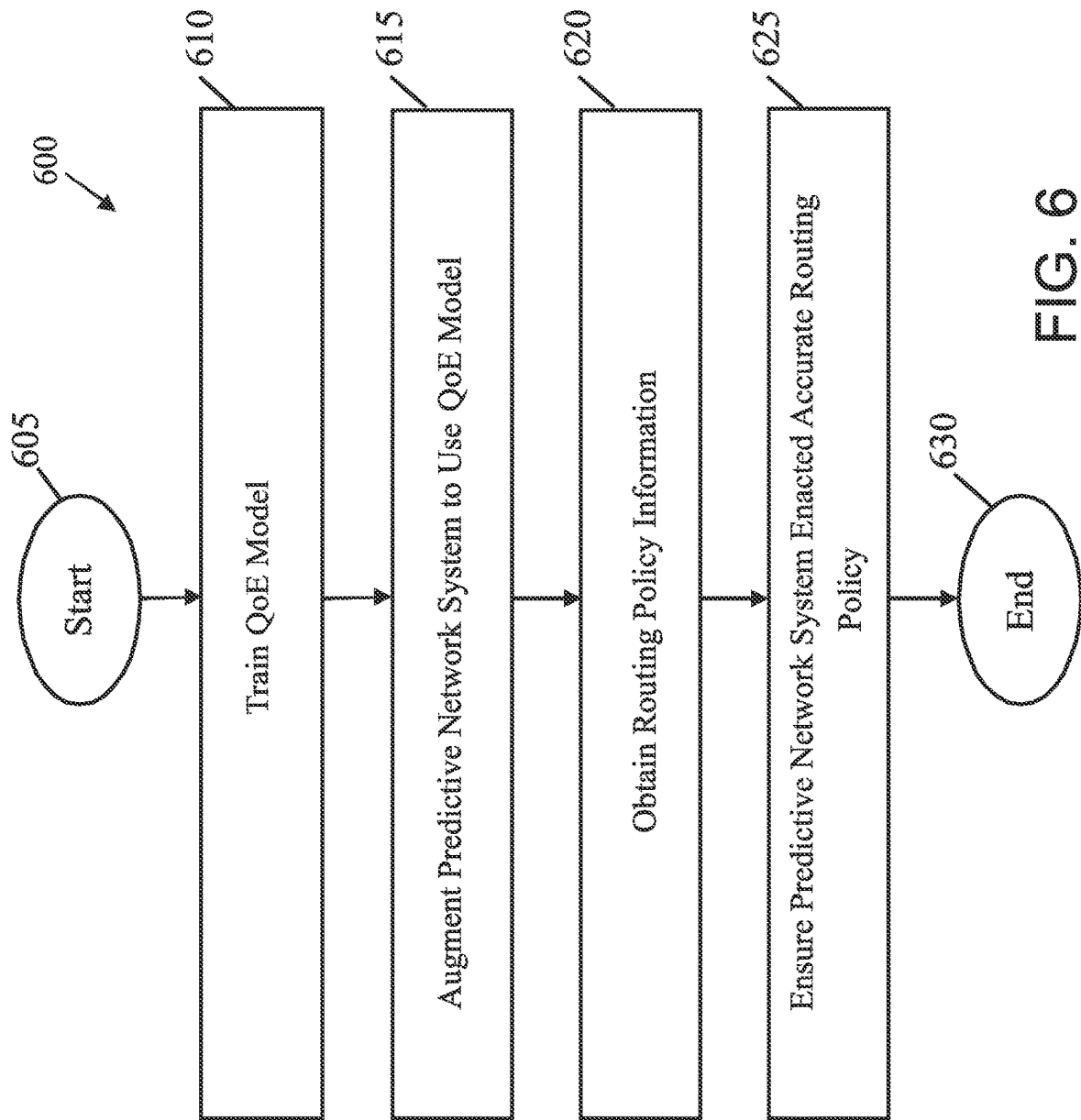


FIG. 6

QOE-DRIVEN PREDICTIVE NETWORKS

TECHNICAL FIELD

[0001] The present disclosure relates generally to computer networks, and, more particularly, to quality of experience (QoE)-driven predictive networks.

BACKGROUND

[0002] Traditionally, network administrators have used service level agreements (SLAs) as a proxy for the quality of experience (QoE) of online applications from the standpoint of their users. These SLAs take the form of thresholds for various network characteristics, such as delay, loss, jitter, etc., and are viewed as the dividing lines between acceptable application experience or degraded application experience. For instance, in the case of voice applications, the usual SLA boundaries are 150 ms for delay, 50 ms for jitter, and maximum of 3% packet loss. This is typically done for each class/type of application of interest (e.g., videoconferencing, audio, etc.).

[0003] With the recent advancements in machine learning, it now becomes possible to produce recommendations that optimize an estimate of the probability that users of a given application will have a good experience within the application, based on predictions as to whether a given network path is likely to exhibit an SLA violation. These so-called predictive network systems are able to take corrective measures such as rerouting the application traffic proactively, in advance of a predicted SLA violation. However, predictive network systems that rely on SLA violations as a proxy for the true QoE of an application also fail to take into account other factors like the application itself, its users, etc. A paradigm shift towards predicting the true QoE of an application is now possible, but the prior deployment of these predictive network systems may be an impediment to adopting such technologies.

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] The embodiments herein may be better understood by referring to the following description in conjunction with the accompanying drawings in which like reference numerals indicate identically or functionally similar elements, of which:

[0005] FIGS. 1A-1B illustrate an example communication network;

[0006] FIG. 2 illustrates an example network device/node;

[0007] FIGS. 3A-3B illustrate example network deployments;

[0008] FIGS. 4A-4B illustrate example software defined network (SDN) implementations;

[0009] FIG. 5 illustrates an example architecture for quality of experience (QoE)-driven predictive networks; and

[0010] FIG. 6 illustrates an example simplified procedure for implementing a QoE-driven predictive network.

DESCRIPTION OF EXAMPLE EMBODIMENTS

Overview

[0011] According to one or more embodiments of the disclosure, a device trains a quality of experience model to predict a quality of experience metric for an online application. The device augments a predictive network system previously deployed to a network to use the quality of

experience model instead of a prediction model that predicts performance of a network path. The device obtains policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model. The device ensures, based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application.

Description

[0012] A computer network is a geographically distributed collection of nodes interconnected by communication links and segments for transporting data between end nodes, such as personal computers and workstations, or other devices, such as sensors, etc. Many types of networks are available, with the types ranging from local area networks (LANs) to wide area networks (WANs). LANs typically connect the nodes over dedicated private communications links located in the same general physical location, such as a building or campus. WANs, on the other hand, typically connect geographically dispersed nodes over long-distance communications links, such as common carrier telephone lines, optical lightpaths, synchronous optical networks (SONET), or synchronous digital hierarchy (SDH) links, or Powerline Communications (PLC) such as IEEE 61334, IEEE P1901.2, and others. The Internet is an example of a WAN that connects disparate networks throughout the world, providing global communication between nodes on various networks. The nodes typically communicate over the network by exchanging discrete frames or packets of data according to predefined protocols, such as the Transmission Control Protocol/Internet Protocol (TCP/IP). In this context, a protocol consists of a set of rules defining how the nodes interact with each other. Computer networks may be further interconnected by an intermediate network node, such as a router, to extend the effective “size” of each network.

[0013] Smart object networks, such as sensor networks, in particular, are a specific type of network having spatially distributed autonomous devices such as sensors, actuators, etc., that cooperatively monitor physical or environmental conditions at different locations, such as, e.g., energy/power consumption, resource consumption (e.g., water/gas/etc. for advanced metering infrastructure or “AMI” applications) temperature, pressure, vibration, sound, radiation, motion, pollutants, etc. Other types of smart objects include actuators, e.g., responsible for turning on/off an engine or perform any other actions. Sensor networks, a type of smart object network, are typically shared-media networks, such as wireless or PLC networks. That is, in addition to one or more sensors, each sensor device (node) in a sensor network may generally be equipped with a radio transceiver or other communication port such as PLC, a microcontroller, and an energy source, such as a battery. Often, smart object networks are considered field area networks (FANs), neighborhood area networks (NANs), personal area networks (PANs), etc. Generally, size and cost constraints on smart object nodes (e.g., sensors) result in corresponding constraints on resources such as energy, memory, computational speed and bandwidth.

[0014] FIG. 1A is a schematic block diagram of an example computer network 100 illustratively comprising nodes/devices, such as a plurality of routers/devices interconnected by links or networks, as shown. For example,

customer edge (CE) routers **110** may be interconnected with provider edge (PE) routers **120** (e.g., PE-1, PE-2, and PE-3) in order to communicate across a core network, such as an illustrative network backbone **130**. For example, routers **110**, **120** may be interconnected by the public Internet, a multiprotocol label switching (MPLS) virtual private network (VPN), or the like. Data packets **140** (e.g., traffic/messages) may be exchanged among the nodes/devices of the computer network **100** over links using predefined network communication protocols such as the Transmission Control Protocol/Internet Protocol (TCP/IP), User Datagram Protocol (UDP), Asynchronous Transfer Mode (ATM) protocol, Frame Relay protocol, or any other suitable protocol. Those skilled in the art will understand that any number of nodes, devices, links, etc. may be used in the computer network, and that the view shown herein is for simplicity.

[0015] In some implementations, a router or a set of routers may be connected to a private network (e.g., dedicated leased lines, an optical network, etc.) or a virtual private network (VPN), such as an MPLS VPN thanks to a carrier network, via one or more links exhibiting very different network and service level agreement characteristics. For the sake of illustration, a given customer site may fall under any of the following categories:

[0016] 1.) Site Type A: a site connected to the network (e.g., via a private or VPN link) using a single CE router and a single link, with potentially a backup link (e.g., a 3G/4G/5G/LTE backup connection). For example, a particular CE router **110** shown in network **100** may support a given customer site, potentially also with a backup link, such as a wireless connection.

[0017] 2.) Site Type B: a site connected to the network by the CE router via two primary links (e.g., from different Service Providers), with potentially a backup link (e.g., a 3G/4G/5G/LTE connection). A site of type B may itself be of different types:

[0018] 2a.) Site Type B1: a site connected to the network using two MPLS VPN links (e.g., from different Service Providers), with potentially a backup link (e.g., a 3G/4G/5G/LTE connection).

[0019] 2b.) Site Type B2: a site connected to the network using one MPLS VPN link and one link connected to the public Internet, with potentially a backup link (e.g., a 3G/4G/5G/LTE connection). For example, a particular customer site may be connected to network **100** via PE-3 and via a separate Internet connection, potentially also with a wireless backup link.

[0020] 2c.) Site Type B3: a site connected to the network using two links connected to the public Internet, with potentially a backup link (e.g., a 3G/4G/5G/LTE connection).

[0021] Notably, MPLS VPN links are usually tied to a committed service level agreement, whereas Internet links may either have no service level agreement at all or a loose service level agreement (e.g., a “Gold Package” Internet service connection that guarantees a certain level of performance to a customer site).

[0022] 3.) Site Type C: a site of type B (e.g., types B1, B2 or B3) but with more than one CE router (e.g., a first CE router connected to one link while a second CE router is connected to the other link), and potentially a backup link (e.g., a wireless 3G/4G/5G/LTE backup link). For example, a particular customer site may

include a first CE router **110** connected to PE-2 and a second CE router **110** connected to PE-3.

[0023] FIG. 1B illustrates an example of network **100** in greater detail, according to various embodiments. As shown, network backbone **130** may provide connectivity between devices located in different geographical areas and/or different types of local networks. For example, network **100** may comprise local/branch networks **160**, **162** that include devices/nodes **10-16** and devices/nodes **18-20**, respectively, as well as a data center/cloud environment **150** that includes servers **152-154**. Notably, local networks **160-162** and data center/cloud environment **150** may be located in different geographic locations.

[0024] Servers **152-154** may include, in various embodiments, a network management server (NMS), a dynamic host configuration protocol (DHCP) server, a constrained application protocol (CoAP) server, an outage management system (OMS), an application policy infrastructure controller (APIC), an application server, etc. As would be appreciated, network **100** may include any number of local networks, data centers, cloud environments, devices/nodes, servers, etc.

[0025] In some embodiments, the techniques herein may be applied to other network topologies and configurations. For example, the techniques herein may be applied to peering points with high-speed links, data centers, etc.

[0026] According to various embodiments, a software-defined WAN (SD-WAN) may be used in network **100** to connect local network **160**, local network **162**, and data center/cloud environment **150**. In general, an SD-WAN uses a software defined networking (SDN)-based approach to instantiate tunnels on top of the physical network and control routing decisions, accordingly. For example, as noted above, one tunnel may connect router CE-2 at the edge of local network **160** to router CE-1 at the edge of data center/cloud environment **150** over an MPLS or Internet-based service provider network in backbone **130**. Similarly, a second tunnel may also connect these routers over a 4G/5G/LTE cellular service provider network. SD-WAN techniques allow the WAN functions to be virtualized, essentially forming a virtual connection between local network **160** and data center/cloud environment **150** on top of the various underlying connections. Another feature of SD-WAN is centralized management by a supervisory service that can monitor and adjust the various connections, as needed.

[0027] FIG. 2 is a schematic block diagram of an example node/device **200** (e.g., an apparatus) that may be used with one or more embodiments described herein, e.g., as any of the computing devices shown in FIGS. 1A-1B, particularly the PE routers **120**, CE routers **110**, nodes/device **10-20**, servers **152-154** (e.g., a network controller/supervisory service located in a data center, etc.), any other computing device that supports the operations of network **100** (e.g., switches, etc.), or any of the other devices referenced below. The device **200** may also be any other suitable type of device depending upon the type of network architecture in place, such as IoT nodes, etc. Device **200** comprises one or more network interfaces **210**, one or more processors **220**, and a memory **240** interconnected by a system bus **250**, and is powered by a power supply **260**.

[0028] The network interfaces **210** include the mechanical, electrical, and signaling circuitry for communicating data over physical links coupled to the network **100**. The

network interfaces may be configured to transmit and/or receive data using a variety of different communication protocols. Notably, a physical network interface **210** may also be used to implement one or more virtual network interfaces, such as for virtual private network (VPN) access, known to those skilled in the art.

[0029] The memory **240** comprises a plurality of storage locations that are addressable by the processor(s) **220** and the network interfaces **210** for storing software programs and data structures associated with the embodiments described herein. The processor **220** may comprise necessary elements or logic adapted to execute the software programs and manipulate the data structures **245**. An operating system **242** (e.g., the Internetworking Operating System, or IOS®, of Cisco Systems, Inc., another operating system, etc.), portions of which are typically resident in memory **240** and executed by the processor(s), functionally organizes the node by, inter alia, invoking network operations in support of software processors and/or services executing on the device. These software processors and/or services may comprise an application experience optimization process **248**, as described herein, any of which may alternatively be located within individual network interfaces.

[0030] It will be apparent to those skilled in the art that other processor and memory types, including various computer-readable media, may be used to store and execute program instructions pertaining to the techniques described herein. Also, while the description illustrates various processes, it is expressly contemplated that various processes may be embodied as modules configured to operate in accordance with the techniques herein (e.g., according to the functionality of a similar process). Further, while processes may be shown and/or described separately, those skilled in the art will appreciate that processes may be routines or modules within other processes.

[0031] In general, application experience optimization process **248** may include computer executable instructions executed by the processor **220** to perform routing functions in conjunction with one or more routing protocols. These functions may, on capable devices, be configured to manage a routing/forwarding table (a data structure **245**) containing, e.g., data used to make routing/forwarding decisions. In various cases, connectivity may be discovered and known, prior to computing routes to any destination in the network, e.g., link state routing such as Open Shortest Path First (OSPF), or Intermediate-System-to-Intermediate-System (ISIS), or Optimized Link State Routing (OLSR). For instance, paths may be computed using a shortest path first (SPF) or constrained shortest path first (CSPF) approach. Conversely, neighbors may first be discovered (e.g., a priori knowledge of network topology is not known) and, in response to a needed route to a destination, send a route request into the network to determine which neighboring node may be used to reach the desired destination. Example protocols that take this approach include Ad-hoc On-demand Distance Vector (AODV), Dynamic Source Routing (DSR), DYnamic MANET On-demand Routing (DYMO), etc. Notably, on devices not capable or configured to store routing entries, application experience optimization process **248** may consist solely of providing mechanisms necessary for source routing techniques. That is, for source routing, other devices in the network can tell the less capable devices

exactly where to send the packets, and the less capable devices simply forward the packets as directed.

[0032] In various embodiments, as detailed further below, application experience optimization process **248** may include computer executable instructions that, when executed by processor(s) **220**, cause device **200** to perform the techniques described herein. To do so, in some embodiments, application experience optimization process **248** may utilize machine learning. In general, machine learning is concerned with the design and the development of techniques that take as input empirical data (such as network statistics and performance indicators) and recognize complex patterns in these data. One very common pattern among machine learning techniques is the use of an underlying model M , whose parameters are optimized for minimizing the cost function associated to M , given the input data. For instance, in the context of classification, the model M may be a straight line that separates the data into two classes (e.g., labels) such that $M=a*x+b*y+c$ and the cost function would be the number of misclassified points. The learning process then operates by adjusting the parameters a, b, c such that the number of misclassified points is minimal. After this optimization phase (or learning phase), the model M can be used very easily to classify new data points. Often, M is a statistical model, and the cost function is inversely proportional to the likelihood of M , given the input data.

[0033] In various embodiments, application experience optimization process **248** may employ one or more supervised, unsupervised, or semi-supervised machine learning models. Generally, supervised learning entails the use of a training set of data, as noted above, that is used to train the model to apply labels to the input data. For example, the training data may include sample telemetry that has been labeled as being indicative of an acceptable performance or unacceptable performance. On the other end of the spectrum are unsupervised techniques that do not require a training set of labels. Notably, while a supervised learning model may look for previously seen patterns that have been labeled as such, an unsupervised model may instead look to whether there are sudden changes or patterns in the behavior of the metrics. Semi-supervised learning models take a middle ground approach that uses a greatly reduced set of labeled training data.

[0034] Example machine learning techniques that application experience optimization process **248** can employ may include, but are not limited to, nearest neighbor (NN) techniques (e.g., k-NN models, replicator NN models, etc.), statistical techniques (e.g., Bayesian networks, etc.), clustering techniques (e.g., k-means, mean-shift, etc.), neural networks (e.g., reservoir networks, artificial neural networks, etc.), support vector machines (SVMs), generative adversarial networks (GANs), long short-term memory (LSTM), logistic or other regression, Markov models or chains, principal component analysis (PCA) (e.g., for linear models), singular value decomposition (SVD), multi-layer perceptron (MLP) artificial neural networks (ANNs) (e.g., for non-linear models), replicating reservoir networks (e.g., for non-linear models, typically for timeseries), random forest classification, or the like.

[0035] In further implementations, application experience optimization process **248** may also include one or more generative artificial intelligence/machine learning models. In contrast to discriminative models that simply seek to perform pattern matching for purposes such as anomaly

detection, classification, or the like, generative approaches instead seek to generate new content or other data (e.g., audio, video/images, text, etc.), based on an existing body of training data. For instance, in the context of network assurance, application experience optimization process 248 may use a generative model to generate synthetic network traffic based on existing user traffic to test how the network reacts. Example generative approaches can include, but are not limited to, generative adversarial networks (GANs), large language models (LLMs), other transformer models, and the like.

[0036] The performance of a machine learning model can be evaluated in a number of ways based on the number of true positives, false positives, true negatives, and/or false negatives of the model. For example, consider the case of a model that predicts whether the QoS of a path will satisfy the service level agreement (SLA) of the traffic on that path. In such a case, the false positives of the model may refer to the number of times the model incorrectly predicted that the QoS of a particular network path will not satisfy the SLA of the traffic on that path. Conversely, the false negatives of the model may refer to the number of times the model incorrectly predicted that the QoS of the path would be acceptable. True negatives and positives may refer to the number of times the model correctly predicted acceptable path performance or an SLA violation, respectively. Related to these measurements are the concepts of recall and precision. Generally, recall refers to the ratio of true positives to the sum of true positives and false negatives, which quantifies the sensitivity of the model. Similarly, precision refers to the ratio of true positives the sum of true and false positives.

[0037] As noted above, in software defined WANs (SD-WANs), traffic between individual sites are sent over tunnels. The tunnels are configured to use different switching fabrics, such as MPLS, Internet, 4G or 5G, etc. Often, the different switching fabrics provide different QoS at varied costs. For example, an MPLS fabric typically provides high QoS when compared to the Internet, but is also more expensive than traditional Internet. Some applications requiring high QoS (e.g., video conferencing, voice calls, etc.) are traditionally sent over the more costly fabrics (e.g., MPLS), while applications not needing strong guarantees are sent over cheaper fabrics, such as the Internet.

[0038] Traditionally, network policies map individual applications to Service Level Agreements (SLAs), which define the satisfactory performance metric(s) for an application, such as loss, latency, or jitter. Similarly, a tunnel is also mapped to the type of SLA that it satisfies, based on the switching fabric that it uses. During runtime, the SD-WAN edge router then maps the application traffic to an appropriate tunnel. Currently, the mapping of SLAs between applications and tunnels is performed manually by an expert, based on their experiences and/or reports on the prior performances of the applications and tunnels.

[0039] The emergence of infrastructure as a service (IaaS) and software-as-a-service (SaaS) is having a dramatic impact of the overall Internet due to the extreme virtualization of services and shift of traffic load in many large enterprises. Consequently, a branch office or a campus can trigger massive loads on the network.

[0040] FIGS. 3A-3B illustrate example network deployments 300, 310, respectively. As shown, a router 110 located at the edge of a remote site 302 may provide connectivity between a local area network (LAN) of the remote site 302

and one or more cloud-based, SaaS providers 308. For example, in the case of an SD-WAN, router 110 may provide connectivity to SaaS provider(s) 308 via tunnels across any number of networks 306. This allows clients located in the LAN of remote site 302 to access cloud applications (e.g., Office 365™, Dropbox™, etc.) served by SaaS provider(s) 308.

[0041] As would be appreciated, SD-WANs allow for the use of a variety of different pathways between an edge device and an SaaS provider. For example, as shown in example network deployment 300 in FIG. 3A, router 110 may utilize two Direct Internet Access (DIA) connections to connect with SaaS provider(s) 308. More specifically, a first interface of router 110 (e.g., a network interface 210, described previously), Int 1, may establish a first communication path (e.g., a tunnel) with SaaS provider(s) 308 via a first Internet Service Provider (ISP) 306a, denoted ISP 1 in FIG. 3A. Likewise, a second interface of router 110, Int 2, may establish a backhaul path with SaaS provider(s) 308 via a second ISP 306b, denoted ISP 2 in FIG. 3A.

[0042] FIG. 3B illustrates another example network deployment 310 in which Int 1 of router 110 at the edge of remote site 302 establishes a first path to SaaS provider(s) 308 via ISP 1 and Int 2 establishes a second path to SaaS provider(s) 308 via a second ISP 306b. In contrast to the example in FIG. 3A, Int 3 of router 110 may establish a third path to SaaS provider(s) 308 via a private corporate network 306c (e.g., an MPLS network) to a private data center or regional hub 304 which, in turn, provides connectivity to SaaS provider(s) 308 via another network, such as a third ISP 306d.

[0043] Regardless of the specific connectivity configuration for the network, a variety of access technologies may be used (e.g., ADSL, 4G, 5G, etc.) in all cases, as well as various networking technologies (e.g., public Internet, MPLS (with or without strict SLA), etc.) to connect the LAN of remote site 302 to SaaS provider(s) 308. Other deployments scenarios are also possible, such as using Colo, accessing SaaS provider(s) 308 via Zscaler or Umbrella services, and the like.

[0044] FIG. 4A illustrates an example SDN implementation 400, according to various embodiments. As shown, there may be a LAN core 402 at a particular location, such as remote site 302 shown previously in FIGS. 3A-3B. Connected to LAN core 402 may be one or more routers that form an SD-WAN service point 406 which provides connectivity between LAN core 402 and SD-WAN fabric 404. For instance, SD-WAN service point 406 may comprise routers 110a-110b.

[0045] Overseeing the operations of routers 110a-110b in SD-WAN service point 406 and SD-WAN fabric 404 may be an SDN controller 408. In general, SDN controller 408 may comprise one or more devices (e.g., a device 200) configured to provide a supervisory service, typically hosted in the cloud, to SD-WAN service point 406 and SD-WAN fabric 404. For instance, SDN controller 408 may be responsible for monitoring the operations thereof, promulgating policies (e.g., security policies, etc.), installing or adjusting IPsec routes/tunnels between LAN core 402 and remote destinations such as regional hub 304 and/or SaaS provider(s) 308 in FIGS. 3A-3B, and the like.

[0046] As noted above, a primary networking goal may be to design and optimize the network to satisfy the requirements of the applications that it supports. So far, though, the

two worlds of “applications” and “networking” have been fairly siloed. More specifically, the network is usually designed in order to provide the best SLA in terms of performance and reliability, often supporting a variety of Class of Service (CoS), but unfortunately without a deep understanding of the actual application requirements. On the application side, the networking requirements are often poorly understood even for very common applications such as voice and video for which a variety of metrics have been developed over the past two decades, with the hope of accurately representing the Quality of Experience (QoE) from the standpoint of the users of the application.

[0047] More and more applications are moving to the cloud and many do so by leveraging an SaaS model. Consequently, the number of applications that became network-centric has grown approximately exponentially with the raise of SaaS applications, such as Office 365, ServiceNow, SAP, voice, and video, to mention a few. All of these applications rely heavily on private networks and the Internet, bringing their own level of dynamicity with adaptive and fast changing workloads. On the network side, SD-WAN provides a high degree of flexibility allowing for efficient configuration management using SDN controllers with the ability to benefit from a plethora of transport access (e.g., MPLS, Internet with supporting multiple CoS, LTE, satellite links, etc.), multiple classes of service and policies to reach private and public networks via multi-cloud SaaS.

[0048] Furthermore, the level of dynamicity observed in today’s network has never been so high. Millions of paths across thousands of Service Providers (SPs) and a number of SaaS applications have shown that the overall QoS(s) of the network in terms of delay, packet loss, jitter, etc. drastically vary with the region, SP, access type, as well as over time with high granularity. The immediate consequence is that the environment is highly dynamic due to:

[0049] New in-house applications being deployed;

[0050] New SaaS applications being deployed everywhere in the network, hosted by a number of different cloud providers;

[0051] Internet, MPLS, LTE transports providing highly varying performance characteristics, across time and regions;

[0052] SaaS applications themselves being highly dynamic: it is common to see new servers deployed in the network. DNS resolution allows the network for being informed of a new server deployed in the network leading to a new destination and a potentially shift of traffic towards a new destination without being even noticed.

[0053] According to various embodiments, application aware routing usually refers to the ability to route traffic so as to satisfy the requirements of the application, as opposed to exclusively relying on the (constrained) shortest path to reach a destination IP address. Various attempts have been made to extend the notion of routing, CSPF, link state routing protocols (ISIS, OSPF, etc.) using various metrics (e.g., Multi-topology Routing) where each metric would reflect a different path attribute (e.g., delay, loss, latency, etc.), but each time with a static metric. At best, current approaches rely on SLA templates specifying the application requirements so as for a given path (e.g., a tunnel) to be “eligible” to carry traffic for the application. In turn, application SLAs are checked using regular probing. Other solutions compute a metric reflecting a particular network

characteristic (e.g., delay, throughput, etc.) and then selecting the supposed ‘best path,’ according to the metric.

[0054] The term ‘SLA failure’ refers to a situation in which the SLA for a given application, often expressed as a function of delay, loss, or jitter, is not satisfied by the current network path for the traffic of a given application. This leads to poor QoE from the standpoint of the users of the application. Modern SaaS solutions like Viptela, CloudonRamp SaaS, and the like, allow for the computation of per application QoE by sending HyperText Transfer Protocol (HTTP) probes along various paths from a branch office and then route the application’s traffic along a path having the best QoE for the application. At a first sight, such an approach may solve many problems. Unfortunately, though, there are several shortcomings to this approach:

[0055] The SLA for the application is ‘guessed,’ using static thresholds.

[0056] Routing is still entirely reactive: decisions are made using probes that reflect the status of a path at a given time, in contrast with the notion of an informed decision.

[0057] SLA failures are very common in the Internet and a good proportion of them could be avoided (e.g., using an alternate path), if predicted in advance.

[0058] In various embodiments, the techniques herein allow for a predictive application aware routing engine to be deployed, such as in the cloud, to control routing decisions in a network. For instance, the predictive application aware routing engine may be implemented as part of an SDN controller (e.g., SDN controller 408) or other supervisory service, or may operate in conjunction therewith. For instance, FIG. 4B illustrates an example 410 in which SDN controller 408 includes a predictive application aware routing engine 412 (e.g., through execution of application experience optimization process 248). Further embodiments provide for predictive application aware routing engine 412 to be hosted on a router 110 or at any other location in the network.

[0059] During execution, predictive application aware routing engine 412 makes use of a high volume of network and application telemetry (e.g., from routers 110a-110b, SD-WAN fabric 404, etc.) so as to compute statistical and/or machine learning models to control the network with the objective of optimizing the application experience and reducing potential down times. To that end, predictive application aware routing engine 412 may compute a variety of models to understand application requirements, and predictably route traffic over private networks and/or the Internet, thus optimizing the application experience while drastically reducing SLA failures and downtimes.

[0060] In other words, predictive application aware routing engine 412 may first predict SLA violations in the network that could affect the QoE of an application (e.g., due to spikes of packet loss or delay, sudden decreases in bandwidth, etc.). In other words, predictive application aware routing engine 412 may use SLA violations as a proxy for actual QoE information (e.g., ratings by users of an online application regarding their perception of the application), unless such QoE information is available from the provider of the online application. In turn, predictive application aware routing engine 412 may then implement a corrective measure, such as rerouting the traffic of the application, prior to the predicted SLA violation. For instance, in the case of video applications, it now becomes

possible to maximize throughput at any given time, which is of utmost importance to maximize the QoE of the video application. Optimized throughput can then be used as a service triggering the routing decision for specific application requiring highest throughput, in one embodiment. In general, routing configuration changes are also referred to herein as routing “patches,” which are typically temporary in nature (e.g., active for a specified period of time) and may also be application-specific (e.g., for traffic of one or more specified applications).

[0061] As noted above, enterprise networks have undergone a fundamental transformation whereby users and applications have become increasingly distributed whereby technologies such as SD-WAN, Hybrid Work, and Zero Trust Network Access (ZTNA) have enabled unprecedented flexibility in terms of network architecture and underlay connectivity options. At the same time, collaboration applications, which are often critical for day-to-day business operations, have moved from on-premises deployment to a SaaS cloud delivery model that allows application vendors to rapidly deploy and take advantage of the latest and greatest codecs that can be used to increase robustness of media content.

[0062] In this highly dynamic environment, the ability of network administrators to understand the impact of network performance (or lack of) on the QoE of online applications, as well as ensuring that the proper SLAs are satisfied, is becoming increasingly challenging. Indeed, in years past, network metrics were used as a proxy for the true application QoE, with SLAs being set, accordingly. For instance, in the case of a voice application, the usual SLA boundaries are 150 ms for delay, 50 ms for jitter, and maximum of 3% packet loss. Today, such values are not as clear-cut. For example, two real-time voice calls may have different loss thresholds based on the audio codec being used whereby a voice application that uses a lossy codec such as Opus may be resistant until a packet loss of up to 30%, whereas other audio codecs, such as advanced audio coding (AAC), are usually not resilient to such high loss thresholds.

[0063] Another factor that demonstrates the shortfalls of relying on SLA thresholds as a proxy for the true application QoE is that SLAs are set without any consideration to the granularity of their underlying measurements. For instance, a path experiencing a constant delay of 120 ms for voice over a period of 10 minutes provides a very different user experience than a path with the same average delay that keeps varying between 20 and 450 ms, despite averaging out to the same over the time period. The dynamics of such metrics is even more critical for packet loss and jitter in the case of voice and video traffic (e.g., ten seconds of 80% packet loss would severely impact the user experience although averaged out over ten minutes would give a low value totally acceptable according to the threshold). Without a doubt, the user experience requires a more subtle and accurate approach in order to determine the networking requirements a path should meet in order to maximize the user satisfaction, capturing local phenomenon (e.g., effects on delay, jitter and loss at higher frequencies) but also telemetry from upper layers (applications).

[0064] Traditionally, a core principle of the Internet has been layer isolation. Such an approach allowed layer dependency (e.g., often referred to as layer violation) to be avoided, at a time where a number of protocols and technologies were developed at each layer. More specifically, the

Open Systems Interconnection (OSI) model divides networks into seven networking layers:

[0065] 1. The Physical (PHY) Layer—the layer representing the physical connections between devices

[0066] 2. The Data Link Layer—e.g., the layer at which MAC addressing is used

[0067] 3. The Network Layer—e.g., the layer at which the IP protocol is used

[0068] 4. The Transport Layer—e.g., the layer at which TCP or UDP

[0069] 5. The Session Layer—e.g., the layer at which a given session between endpoints is managed

[0070] 6. The Presentation Layer—e.g., the layer that translates requests from the application layer to the session layer and vice-versa

[0071] 7. The Application Layer—e.g., the highest layer at which the application itself operates

[0072] This allowed for the design and deployment of new layers (e.g., PHY, MAC, etc.) independent of each other, and allowing the Internet to scale. Still, with modern applications requiring tight SLAs, a cross-layer approach would be highly beneficial to optimizing the QoE of any given online application.

[0073] Further, even with a mechanism that is able to accurately estimate the application experience from the perspective of a user, another challenge still exists with respect to selecting the appropriate network action to improve the experience. Indeed, although the effect of specific actions at a given layer of the networking stack on user experience can be qualitatively evaluated, being able to precisely quantify it is often unknown. For instance, determining that voice quality is low along a highly congested path may be relatively easy. However, determining the correct amount of bandwidth to allocate to the path or the appropriate queue weight for the traffic of the application remains quite challenging.

[0074] According to various embodiments, application experience optimization process 248 may leverage the concept of cognitive networking. Instead of taking a siloed approach where networking systems poorly understand user satisfaction, cognitive networks are fully driven by understanding user experience (cognition) using cross-layer telemetry and ground truth user feedback, in order to determine which networking actions can optimize the user experience. To that end, a rich set of telemetry sources are gathered along with labeled user feedback in order to train a machine learning model to predict/forecast the user experience (i.e., the QoE of an online application). Such a holistic approach that is end-to-end across the different network layers is a paradigm shift to how networks have been designed and operated since the early days of the Internet.

[0075] As noted above, cognitive networks represent an evolution over existing networking techniques by focusing on the true user experience of an online application, rather than attempting to infer this information from proxy information, such as SLA violations (e.g., real SLA violations detected in the network or SLA violations predicted by a predictive network system). The scope of cognitive networks is also not specific to just voice and video applications and can be expanded to other types of applications, as well. Of particular interest are web applications, where users interact with the application directly from a web browser and without the need for dedicated software, which is a popular way for users to interact with many SaaS services.

[0076] However, extending cognitive networking capabilities to existing predictive network systems (PNS) remains challenging. Indeed, these types of systems are capable of producing recommendations that optimize an estimate of the probability that users of a given application have a good experience when using the application. However, such predictions are also based on static thresholds in terms of loss, latency, and jitter (i.e., SLA thresholds). What is needed is a mechanism to augment existing PNS with cognitive networking capabilities, which consider both application-level and network-level telemetry to predict the actual fraction of user satisfaction based on real user feedback, which can be captured offline from paid workers using a tool like Mechanical Turk, or online using real users of the application that provide feedback with the application, via a chatbot, or the like.

Quality of Experience (QoE)-Driven Predictive Networks

[0077] The techniques herein introduce several mechanisms that allow for the performance and capabilities of an existing predictive network system (PNS) to be improved through use of cognitive networking. In some aspects, the techniques herein do so by constructing quality of experience (QoE) models that take into account telemetry across the various layers, including telemetry from the application itself and from the network.

[0078] Illustratively, the techniques described herein may be performed by hardware, software, and/or firmware, such as in accordance with application experience optimization process 248, which may include computer executable instructions executed by the processor 220 (or independent processor of interfaces 210) to perform functions relating to the techniques described herein.

[0079] Specifically, according to various embodiments, a device trains a quality of experience model to predict a quality of experience metric for an online application. The device augments a predictive network system previously deployed to a network to use the quality of experience model instead of a prediction model that predicts performance of a network path. The device obtains policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model. The device ensures, based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application.

[0080] Operationally, FIG. 5 illustrates an example architecture 500 for quality of experience (QoE)-driven predictive networks, according to various embodiments. At the core of architecture 500 is application experience optimization process 248, which may be executed by a controller for a network or another device in communication therewith. For instance, application experience optimization process 248 may be executed by a controller for a network (e.g., SDN controller 408 in FIGS. 4A-4B, such as part of predictive application aware routing engine 412), a particular networking device in the network (e.g., a router, a firewall, etc.), a server, another device or service in communication therewith, or the like.

[0081] As shown, application experience optimization process 248 may include any or all of the following components: a QoE model generator 502, a path selector 504, a

user activity estimator 506, and/or a policy analyzer 508. As would be appreciated, the functionalities of these components may be combined or omitted, as desired. In addition, these components may be implemented on a singular device or in a distributed manner, in which case the combination of executing devices can be viewed as their own singular device for purposes of executing application experience optimization process 248.

[0082] During execution, application experience optimization process 248 may interact with a deployed predictive network system 510, to enhance it with cognitive network capabilities. In further implementations, application experience optimization process 248 may be integrated directly into predictive network system 510 (e.g., ThousandEyes WAN Insights or vAnalytics by Cisco Systems, Inc., etc.). In general, predictive network system 510 may be configured to predict network SLA violations (e.g., the path quality), to infer situations in which the application experience is degraded for users of an online application. In some implementations, predictive network system 510 may also be configured to generate corrective measures, as well, such as by rerouting the application traffic in the network, sending alerts, or the like. In some cases, predictive network system 510 may also be configured initially such that its predictions do not take into account any actual user feedback from users of the online application, instead attempting to infer their degrees of satisfaction based solely on the path performance predictions alone.

[0083] In various implementations, application experience optimization process 248 may obtain any or all of the following types of information:

[0084] Network telemetry 512—such telemetry may be generated by one or more routers or other networking devices in the network (e.g., the CE router associated with a given endpoint client, etc.), an agent on the endpoint itself, or any other device in the network, and indicate performance metrics such as path loss, latency, jitter, etc. In various embodiments, network telemetry 512 may also include traceroute information captured by agents (e.g., ThousandEyes agents, etc.) executed by these devices performing path tracing/probing.

[0085] Application telemetry 514—In addition to obtaining network telemetry 512, application experience optimization process 248 may also obtain telemetry data generated by the online application of interest, itself. For instance, such telemetry may indicate the application experiences of its users, such as user satisfaction ratings (e.g., as collected within the application, collected by a bot separate from the application, etc.), application-level metrics that could indicate the true QoE of the application (e.g., concealment rate, etc.), or other such information.

[0086] QoE model generator 502 may be responsible for performing QoE estimation over a sufficiently long period of history of network telemetry 512 and application telemetry 514, in order to build a QoE prediction model for use by predictive network system 510 for one or more online applications. As noted above, predictive network system 510 may take as input a timeseries of the so-called path quality, which is a proxy or an estimate of the probability that users of a given application have a good experience. In a predictive network system 510 such as ThousandEyes WAN Insights, this quality is computed from Bidirectional Forwarding Detection (BFD) or HTTP probes (coming from

SD-WAN controllers). In various implementations, the QoE model generated by QoE model generator **502** may serve as a drop-in replacement of the component of predictive network system **510** that computes the quality score.

[0087] In one embodiment, QoE model generator **502** may generate a QoE model that relies solely on network telemetry **512** (e.g., Layer 3 metrics) to predict the QoE of an application. In further embodiments, QoE model generator **502** may generate a QoE model that leverages application telemetry **514** (e.g., Layer 7 metrics such as media quality reports from Webex, Zoom, or Teams, etc.), to perform its predictions. Regardless of the embodiment, predictive network system **510** may then pass the predictions of the QoE model to its forecasting engine, which will treat those as any other path-wise quality score.

[0088] However, a challenge with application/Layer 7 telemetry is that it is available only when traffic flows along a given path. To circumvent this problem, various implementations are possible:

[0089] QoE model generator **502** building two distinct models: one that relies on Layer 3 metrics/network telemetry **512** only, and a second model that relies on both Layer 3 metrics/network telemetry **512** and Layer 7 metrics/application telemetry **514**.

[0090] QoE model generator **502** using domain adaptation techniques to build a model that can predict in both domains (e.g., Layer 3 and Layer 7, or Layer 3 and Layer 3+Layer 7).

[0091] Now, predictive network system **510** may use QoE scores from the Layer 3-only model to estimate the quality of paths for which no traffic is present. This is often the case of paths configured as alternates in a SD-WAN network.

[0092] Path selector **504** may leverage explainable AI (XAI) techniques to improve the path selection by predictive network system **510**, especially to detect and alleviate congestion. Indeed, one of the challenges with state-of-the-art predictive systems, such as WAN Insights, is that they have trouble identifying when an issue is due to a middle-mile problem that could be fixed by a re-routing action, or a first-mile congestion caused by the local traffic. Using a QoE model can help with this type of decisions, as it can be used to compute Shapley values for a given path, to identify key drivers of poor experience. For applications such as voice and video, congestion is clearly identifiable using Layer 7/application telemetry **514**, such as video bitrate or resolution, since codecs automatically reduce the resolution in the case of congestion. To this end, path selector **504** may interact with predictive network system **510** or be integrated directly into it, to aid in the selection of the routing strategy used for the application traffic.

[0093] Now, if a path is subject to first-mile congestion due to the traffic itself, re-routing only a subset of the traffic away from this path might be a better solution than trying to find an alternate path. This strategy significantly increases the potential efficacy of predictive network system **510**, as it may now reroute non-critical traffic (e.g., backups) to an alternate, lower-quality path (e.g., with high latency) to preserve real-time applications such as voice on a primary path. As originally deployed, predictive network system **510** may be unable to produce such a recommendation as it only ever considers moving critical traffic to explicitly better paths, rather than moving non-critical traffic away from paths that they might congest. In yet another embodiment, should the root cause be due to local interface congestion (as

shown by the Shapley values), path selector **504** may retrieve local variables (queue lengths, packet drop), to determine the proportion of traffic with low QoE that should be rerouted onto the alternate path.

[0094] The same capability of path selector **504** can also be used to detect congestion that proceeds from a routing recommendation by predictive network system **510**, revert it, and update an internal database of available bandwidth for each path. As a result, predictive network system **510** may then adapt its future recommendations to account for the bandwidth of all paths in the network.

[0095] In various implementations, user activity estimator **506** may assess the Layer 7/application telemetry **514** consumed by the QoE model generated by QoE model generator **502**, to significantly improve the way predictive network system **510** estimates user activity. Indeed, existing PNS deployed today currently estimates user activity based on IP and port (TCP/UDP) information found in NetFlow-type telemetry, which improves the identification of individual users and whether they are truly active (as opposed to control or machine-generated traffic). However, using the techniques herein, application experience optimization process **248** may augment predictive network system **510** to rely on Layer 7/application telemetry **514**. Such telemetry, though, may also include much more details about the user activity (e.g., endpoint details, active speaker status, meeting identifier for voice/video applications, waterfall-type trace for web-based applications), which can be used by the QoE model generated by QoE model generator **502** to perform QoE inference, as well as by user activity estimator **506** to estimate accurately the number of active users of the online application.

[0096] Finally, policy analyzer **508** may ensure that routing policies enacted by predictive network system **510** match accurately the traffic considered by the QoE model generated by QoE model generator **502**. Indeed, a QoE model trained for Webex should ideally be used only in association with a routing policy that influences Webex traffic (e.g., recognized using NBAR2 or other application detection mechanism). To this end, policy analyzer **508** may query the network controller (e.g., vManage, etc.) to 1.) validate that a matching policy exists, 2.) create one if none is found, and 3.) split an existing policy that is too broad (e.g., because it includes other applications from the same family). Optionally, policy analyzer **508** may optionally, empirically validate the effectiveness of applied policies by comparing Layer 7 telemetry/application telemetry **514** (e.g., MQE Webex reports) and Layer 4 flow telemetry/network telemetry **512** (e.g., NetFlow), and how they respectively change upon enacting a policy. If a mismatch is observed, policy analyzer **508** may report an alert to a user interface associated with the network operator.

[0097] FIG. 6 illustrates an example simplified procedure **600** (e.g., a method) for implementing a QoE-driven predictive network, in accordance with one or more embodiments described herein. For example, a non-generic, specifically configured device (e.g., device **200**), such as a router, firewall, controller for a network (e.g., an SDN controller or other device in communication therewith), server, or the like, may perform procedure **600** by executing stored instructions (e.g., application experience optimization process **248**). The procedure **600** may start at step **605**, and continues to step **610**, where, as described in greater detail above, the device may train a quality of experience model to

predict a quality of experience metric for an online application. In various implementations, the device trains the quality of experience model using feedback from users of the online application (e.g., as captured within the application, by a separate chatbot or other polling process, etc.). In some implementations, the quality of experience model takes as input Layer 3 telemetry obtained from the network. In further implementations, the quality of experience model takes as input Layer 7 telemetry obtained from the online application.

[0098] At step 615, as detailed above, the device may augment a predictive network system previously deployed to a network to use the quality of experience model instead of a prediction model that predicts performance of a network path. In some instances, the prediction model predicts service level agreement (SLA) violations by the network path.

[0099] In some implementations, the quality of experience model is a first quality of experience model and the device also trains a second quality of experience model. In such a case, the device may further augment the predictive network system to switch between using the first quality of experience model and the second quality of experience model, depending on whether there are traffic flows in the network with the online application.

[0100] At step 620, the device may obtain policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model, as described in greater detail above. For instance, the device may retrieve routing policy information from a network controller associated with the predictive network system.

[0101] At step 625, as detailed above, the device ensures, based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application (e.g., the policy does not inadvertently impinge on other traffic not associated with the application, the policy is not too broad and applies to applications from the same family, etc.). In some implementations, the device may also use Layer 7 telemetry from the online application to augment how the predictive network system estimates user activity with respect to the online application. In further implementations, the device may also cause the predictive network system to detect congestion in the network using the quality of experience model. In such a case, the device may further cause the predictive network system to reroute non-critical traffic onto a different path in the network from a path that conveys traffic associated with the online application, when there is detected congestion along the path that conveys traffic associated with the online application.

[0102] Procedure 600 then ends at step 630.

[0103] It should be noted that while certain steps within procedure 600 may be optional as described above, the steps shown in FIG. 6 are merely examples for illustration, and certain other steps may be included or excluded as desired. Further, while a particular order of the steps is shown, this ordering is merely illustrative, and any suitable arrangement of the steps may be utilized without departing from the scope of the embodiments herein.

[0104] While there have been shown and described illustrative embodiments that provide for quality of experience (QoE)-driven predictive networks, it is to be understood that various other adaptations and modifications may be made

within the spirit and scope of the embodiments herein. For example, while certain embodiments are described herein with respect to using certain models for purposes of predicting application experience metrics (e.g., QoE metrics), SLA violations, or other disruptions in a network, the models are not limited as such and may be used for other types of predictions, in other embodiments. In addition, while certain protocols are shown, other suitable protocols may be used, accordingly.

[0105] The foregoing description has been directed to specific embodiments. It will be apparent, however, that other variations and modifications may be made to the described embodiments, with the attainment of some or all of their advantages. For instance, it is expressly contemplated that the components and/or elements described herein can be implemented as software being stored on a tangible (non-transitory) computer-readable medium (e.g., disks/CDs/RAM/EEPROM/etc.) having program instructions executing on a computer, hardware, firmware, or a combination thereof. Accordingly, this description is to be taken only by way of example and not to otherwise limit the scope of the embodiments herein. Therefore, it is the object of the appended claims to cover all such variations and modifications as come within the true spirit and scope of the embodiments herein.

1. A method comprising:

training, by a device, a quality of experience model to predict a quality of experience metric for an online application;

augmenting, by the device, a predictive network system previously deployed to a network to use the quality of experience model instead of a prediction model that predicts performance of a network path;

obtaining, by the device, policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model; and

ensuring, by the device and based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application.

2. The method as in claim 1, wherein the device trains the quality of experience model using feedback from users of the online application.

3. The method as in claim 1, wherein the quality of experience model takes as input Layer 3 telemetry obtained from the network.

4. The method as in claim 3, wherein the quality of experience model takes as input Layer 7 telemetry obtained from the online application.

5. The method as in claim 1, wherein the quality of experience model is a first quality of experience model, the method further comprising:

training, by the device, a second quality of experience model; and

augmenting, by the device, the predictive network system to switch between using the first quality of experience model and the second quality of experience model, depending on whether there are traffic flows in the network with the online application.

6. The method as in claim 1, further comprising:
using, by the device, Layer 7 telemetry from the online application to augment how the predictive network system estimates user activity with respect to the online application.
7. The method as in claim 1, further comprising:
causing, by the device, the predictive network system to detect congestion in the network using the quality of experience model.
8. The method as in claim 7, further comprising:
causing, by the device, the predictive network system to reroute non-critical traffic onto a different path in the network from a path that conveys traffic associated with the online application, when there is detected congestion along the path that conveys traffic associated with the online application.
9. The method as in claim 1, wherein the prediction model predicts service level agreement (SLA) violations by the network path.
10. The method as in claim 1, wherein ensuring that the predictive network system enacted a routing policy that accurately matches traffic for the online application comprises:
splitting an existing routing policy or enacting a new routing policy.
11. An apparatus, comprising:
one or more network interfaces;
a processor coupled to the one or more network interfaces and configured to execute one or more processes; and
a memory configured to store a process that is executable by the processor, the process when executed configured to:
train a quality of experience model to predict a quality of experience metric for an online application;
augment a predictive network system previously deployed to a network to use the quality of experience model instead of a prediction model that predicts performance of a network path;
obtain policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model; and
ensure, based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application.
12. The apparatus as in claim 11, wherein the apparatus trains the quality of experience model using feedback from users of the online application.
13. The apparatus as in claim 11, wherein the quality of experience model takes as input Layer 3 telemetry obtained from the network.

14. The apparatus as in claim 13, wherein the quality of experience model takes as input Layer 7 telemetry obtained from the online application.

15. The apparatus as in claim 11, wherein the quality of experience model is a first quality of experience model, wherein the process when executed is further configured to:
train a second quality of experience model; and
augment the predictive network system to switch between using the first quality of experience model and the second quality of experience model, depending on whether there are traffic flows in the network with the online application.

16. The apparatus as in claim 11, wherein the process when executed is further configured to:

use Layer 7 telemetry from the online application to augment how the predictive network system estimates user activity with respect to the online application.

17. The apparatus as in claim 11, wherein the process when executed is further configured to:

cause the predictive network system to detect congestion in the network using the quality of experience model.

18. The apparatus as in claim 17, wherein the process when executed is further configured to:

cause the predictive network system to reroute non-critical traffic onto a different path in the network from a path that conveys traffic associated with the online application, when there is detected congestion along the path that conveys traffic associated with the online application.

19. The apparatus as in claim 11, wherein the prediction model predicts service level agreement (SLA) violations by the network path.

20. A tangible, non-transitory, computer-readable medium storing program instructions that cause a device to execute a process comprising:

training, by the device, a quality of experience model to predict a quality of experience metric for an online application;

augmenting, by the device, a predictive network system previously deployed to a network to use the quality of experience model instead of a prediction model that predicts performance of a network path;

obtaining, by the device, policy information indicative of whether the predictive network system enacted one or more routing policies in the network based on a prediction by the quality of experience model; and

ensuring, by the device and based on the policy information, that the predictive network system that was augmented to use the quality of experience model enacted a routing policy that accurately matches traffic for the online application.

* * * * *